

Explicación Detallada: Base de Datos `modelado_bd`

Índice

1. [Introducción](#)
 2. [Conceptos Fundamentales: DDL y DML](#)
 3. [Análisis de la Estructura](#)
 4. [Creación de la Base de Datos \(DDL\)](#)
 5. [Definición de Tablas](#)
 6. [Manipulación de Datos \(DML\)](#)
 7. [Relaciones y Integridad Referencial](#)
 8. [Operaciones de Consulta y Modificación](#)
 9. [Buenas Prácticas Observadas](#)
 10. [Posibles Mejoras](#)
-

Introducción

Este documento analiza en detalle la base de datos `modelado_bd`, que implementa un sistema básico de gestión de usuarios con información personal. La base de datos está diseñada siguiendo principios de normalización y utiliza MySQL como sistema gestor de base de datos.

Propósito del Sistema:

- Almacenar información personal de individuos
 - Gestionar cuentas de usuario asociadas a personas
 - Mantener la integridad referencial entre entidades
-

Conceptos Fundamentales: DDL y DML

DDL (Data Definition Language) - Lenguaje de Definición de Datos

El **DDL** es un subconjunto del SQL que se utiliza para definir y modificar la estructura de la base de datos. Sus características principales son:

Propósito:

- Crear, modificar y eliminar objetos de base de datos
- Definir la estructura de tablas, índices, vistas, etc.
- Establecer restricciones y relaciones

Comandos Principales:

- **CREATE**: Crea nuevos objetos (bases de datos, tablas, índices)
- **ALTER**: Modifica la estructura de objetos existentes
- **DROP**: Elimina objetos de la base de datos
- **TRUNCATE**: Elimina todos los datos de una tabla manteniendo su estructura

Características:

- Las operaciones DDL son **auto-commit** (se confirman automáticamente)
- Afectan la estructura, no los datos
- Requieren privilegios especiales para ejecutarse

DML (Data Manipulation Language) - Lenguaje de Manipulación de Datos

El **DML** se utiliza para manipular los datos almacenados en las estructuras creadas con DDL.

Propósito:

- Insertar, consultar, actualizar y eliminar datos
- Realizar operaciones sobre el contenido de las tablas

Comandos Principales:

- **INSERT**: Inserta nuevos registros
- **SELECT**: Consulta y recupera datos
- **UPDATE**: Modifica registros existentes
- **DELETE**: Elimina registros

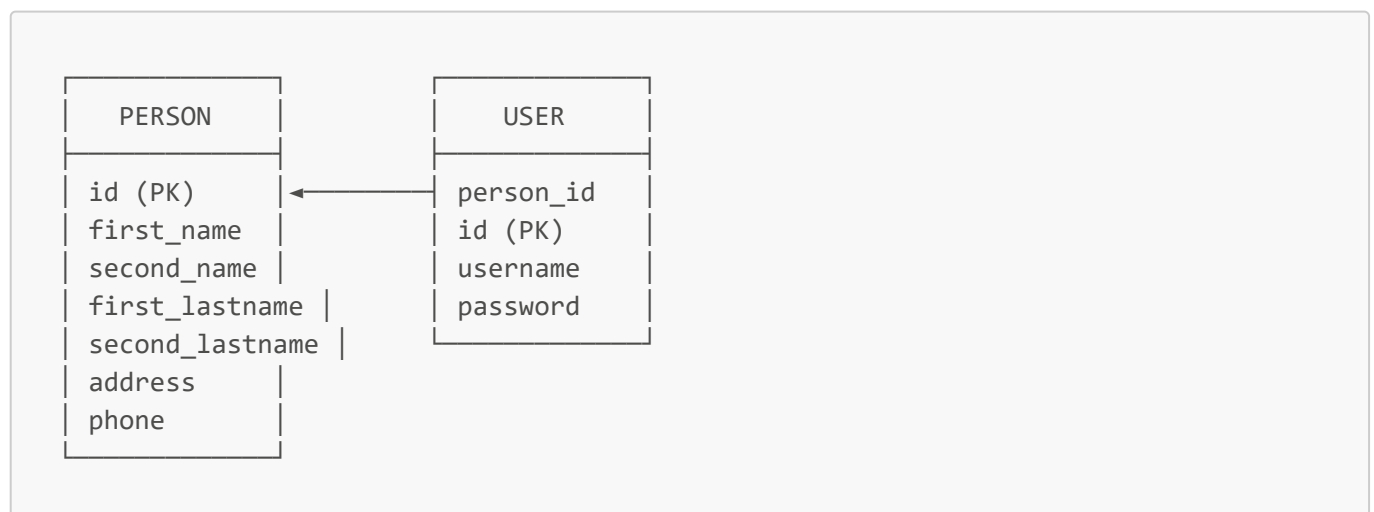
Características:

- Las operaciones pueden ser parte de transacciones
- Trabajan con el contenido, no con la estructura
- Pueden ser reversibles usando ROLLBACK (en transacciones)

Análisis de la Estructura

Modelo Entidad-Relación

El sistema implementa dos entidades principales:



Relación: Una persona puede tener un usuario (1:1 opcional)

Creación de la Base de Datos (DDL)

1. Verificación y Eliminación Preventiva

```
DROP DATABASE IF EXISTS modelado_bd;
```

Explicación:

- **Comando DDL:** `DROP DATABASE`
- **Función:** Elimina la base de datos si existe previamente
- **Cláusula IF EXISTS:** Evita errores si la base de datos no existe
- **Uso:** Garantiza un estado limpio antes de la creación

Beneficios:

- Previene conflictos con bases de datos existentes
- Permite ejecución repetida del script
- Asegura consistencia en el entorno de desarrollo

2. Creación de la Base de Datos

```
CREATE DATABASE modelado_bd;
```

Explicación:

- **Comando DDL:** `CREATE DATABASE`
- **Función:** Crea una nueva base de datos vacía
- **Nombre:** `modelado_bd` (descriptivo del propósito)

3. Selección de la Base de Datos

```
USE modelado_bd;
```

Explicación:

- **Comando:** Establece el contexto de trabajo
- **Función:** Todas las operaciones posteriores se ejecutarán en esta BD
- **Importancia:** Evita especificar el nombre de BD en cada consulta

Definición de Tablas

Tabla `person` (Entidad Principal)

```
CREATE TABLE person(  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    second_name VARCHAR(50) NOT NULL,  
    first_lastname VARCHAR(50) NOT NULL,  
    second_lastname VARCHAR(50) NOT NULL,  
    addres VARCHAR(50) NOT NULL,  
    phone VARCHAR(50) NOT NULL  
);
```

Análisis de Columnas:

Campo	Tipo	Restricciones	Propósito
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único automático
first_name	VARCHAR(50)	NOT NULL	Primer nombre (obligatorio)
second_name	VARCHAR(50)	NOT NULL	Segundo nombre (obligatorio)
first_lastname	VARCHAR(50)	NOT NULL	Primer apellido (obligatorio)
second_lastname	VARCHAR(50)	NOT NULL	Segundo apellido (obligatorio)
addres	VARCHAR(50)	NOT NULL	Dirección (nota: error tipográfico)
phone	VARCHAR(50)	NOT NULL	Teléfono (obligatorio)

Características Técnicas:

Clave Primaria:

- `id INT PRIMARY KEY AUTO_INCREMENT`
- Garantiza unicidad de registros
- Genera valores automáticamente (1, 2, 3...)
- Mejora rendimiento en consultas y relaciones

Restricciones NOT NULL:

- Todos los campos son obligatorios
- Previene datos incompletos
- Asegura integridad de información personal

Tipos de Datos:

- `INT`: Números enteros para el ID
- `VARCHAR(50)`: Cadenas de texto variable hasta 50 caracteres
- Longitud adecuada para nombres y direcciones típicas

Tabla **user** (Entidad Dependiente)

```
CREATE TABLE user(  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(50) NOT NULL,  
  person_id INT,  
  FOREIGN KEY (person_id) REFERENCES person(id)  
);
```

Análisis de Columnas:

Campo	Tipo	Restricciones	Propósito
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único del usuario
username	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de usuario único
password	VARCHAR(50)	NOT NULL	Contraseña (sin encriptar)
person_id	INT	FOREIGN KEY	Referencia a la persona

Características Avanzadas:

Restricción UNIQUE:

- username VARCHAR(50) NOT NULL UNIQUE
- Garantiza que no haya dos usuarios con el mismo nombre
- Facilita autenticación sin ambigüedades

Clave Foránea:

- FOREIGN KEY (person_id) REFERENCES person(id)
- Establece relación con la tabla person
- Mantiene integridad referencial
- person_id puede ser NULL (usuario sin persona asociada)

Manipulación de Datos (DML)

Inserción de Datos

1. Inserción Individual con Formato Extendido

```
INSERT INTO person(  
  first_name,  
  second_name,  
  first_lastname,  
  second_lastname,
```

```

    addres,
    phone
)VALUES(
    'Juan',
    'Manuel',
    'Ortiz',
    'Perez',
    'Calle 21 # 12-A1',
    '32045436456'
);

```

Características:

- **Comando DML:** `INSERT INTO`
- **Formato:** Especifica columnas explícitamente
- **Ventajas:**
 - Más legible y mantenible
 - Independiente del orden de columnas en la tabla
 - Auto-documenta qué valor va en qué campo

2. Inserción Múltiple Compacta

```

INSERT INTO person(first_name, second_name, first_lastname, second_lastname,
addres, phone)VALUES
    ('Adriana', 'María', 'Mendez', '', 'Calle 34 # 01-A1', '4324522343'),
    ('Karol', '', 'Rodroguéz', '', 'Calle 34 # 01-A1', '4324522343');

```

Características:

- **Eficiencia:** Una sola declaración INSERT para múltiples registros
- **Rendimiento:** Más rápido que múltiples INSERT individuales
- **Campos Vacíos:** Usa cadenas vacías (") para campos opcionales

3. Inserción de Usuarios con Referencias

```

INSERT INTO user(
    username,
    password,
    person_id
)VALUES(
    'juan12',
    'helloworld',
    3
);

```

Aspectos Importantes:

- `person_id = 3` referencia al tercer registro de `person`
- Mantiene integridad referencial
- El valor debe existir en la tabla referenciada

Relaciones y Integridad Referencial

Análisis de la Relación Person-User

La base de datos implementa una relación **uno a uno opcional** entre `person` y `user`:

Características de la Relación:

1. **Cardinalidad:** 1:1 (una persona - un usuario máximo)
2. **Opcionalidad:** Una persona puede no tener usuario
3. **Dependencia:** Un usuario debe estar asociado a una persona

Integridad Referencial:

```
FOREIGN KEY (person_id) REFERENCES person(id)
```

Efectos:

- No se puede insertar un usuario con `person_id` inexistente
- No se puede eliminar una persona referenciada por un usuario
- Se mantiene consistencia automáticamente

Ejemplo de Datos Relacionados:

```
person table:
id=1: Juan Manuel Ortiz Perez
id=2: Adriana María Mendez
id=3: Karol Rodríguez

user table:
juan12 → person_id=3 (Karol)
adriana33 → person_id=2 (Adriana)
karol11 → person_id=3 (Karol) ← PROBLEMA: Viola 1:1
```

Operaciones de Consulta y Modificación

Consultas SELECT

```
SELECT * FROM person;
SELECT * FROM user;
```

Análisis:

- **Comando DML:** `SELECT`
- **Operador:** `*` (todos los campos)
- **Propósito:** Verificar datos insertados
- **Uso:** Debugging y validación

Resultado Esperado:

person table:

```
+---+-----+-----+-----+-----+-----+
--+-----+
| id | first_name | second_name | first_lastname | second_lastname | address |
| phone |
+---+-----+-----+-----+-----+-----+
--+-----+
| 1 | Juan      | Manuel     | Ortiz          | Perez           | Calle 21 # 12-
A1| 32045436456 |
| 2 | Adriana  | María      | Mendez         |                  | Calle 34 # 01-
A1| 4324522343 |
| 3 | Carol    | Ximena     | Rodroguéz     |                  | Calle 34 # 01-
A1| 4324522343 |
+---+-----+-----+-----+-----+-----+
--+-----+
```

Actualización de Datos

```
UPDATE person
SET
    first_name = 'Carol',
    second_name = 'Ximena'
WHERE id = 3;
```

Análisis del UPDATE:

- **Comando DML:** `UPDATE`
- **Tabla Objetivo:** `person`
- **Campos Modificados:** `first_name` y `second_name`
- **Condición:** `WHERE id = 3` (especifica un registro)

Proceso de Ejecución:

1. Localiza el registro con `id = 3`
2. Cambia 'Karol' → 'Carol'
3. Cambia '' → 'Ximena'
4. Mantiene otros campos sin cambios

Importancia de WHERE:

- Sin WHERE se actualizarían **todos** los registros
- WHERE asegura modificación específica
- Previene modificaciones accidentales masivas

Eliminación de Datos

```
DELETE FROM user WHERE person_id = 2;  
DELETE FROM person WHERE id = 2;
```

Análisis del DELETE:

Primera Eliminación:

```
DELETE FROM user WHERE person_id = 2;
```

- Elimina usuario asociado a `person_id = 2`
- **Necesario** antes de eliminar la persona
- Respetar integridad referencial

Segunda Eliminación:

```
DELETE FROM person WHERE id = 2;
```

- Elimina persona con `id = 2`
- Solo posible tras eliminar referencias

Orden de Eliminación:

1. **Primero:** Eliminar registros dependientes (`user`)
2. **Segundo:** Eliminar registro principal (`person`)
3. **Razón:** Evitar violación de integridad referencial

Buenas Prácticas Observadas

☒ Aspectos Positivos

1. Uso de Claves Primarias Auto-incrementales

- Garantiza unicidad automática
- Mejora rendimiento
- Simplifica gestión de IDs

2. Implementación de Integridad Referencial

- Foreign Keys mantienen consistencia
- Previene datos huérfanos
- Asegura relaciones válidas

3. Restricciones NOT NULL Apropriadas

- Campos esenciales son obligatorios
- Previene datos incompletos
- Mejora calidad de datos

4. Verificación Preventiva con DROP IF EXISTS

- Permite re-ejecución del script
- Evita errores de duplicación
- Facilita desarrollo iterativo

5. Nombres Descriptivos

- Tablas y campos con nombres claros
- Facilita mantenimiento
- Mejora legibilidad del código

Operaciones DDL Identificadas

1. **DROP DATABASE IF EXISTS** - Eliminación preventiva
2. **CREATE DATABASE** - Creación de base de datos
3. **CREATE TABLE** - Definición de estructuras
4. **PRIMARY KEY** - Definición de claves primarias
5. **FOREIGN KEY** - Definición de relaciones

Operaciones DML Identificadas

1. **INSERT INTO** - Inserción de datos
2. **SELECT** - Consulta de datos
3. **UPDATE** - Modificación de datos
4. **DELETE** - Eliminación de datos

Posibles Mejoras

Mejoras Técnicas

1. Corrección de Errores Tipográficos

```
-- Actual
addres VARCHAR(50) NOT NULL

-- Sugerido
address VARCHAR(50) NOT NULL
```

2. Mejora en Seguridad de Contraseñas

```
-- Actual
password VARCHAR(50) NOT NULL

-- Sugerido
password_hash VARCHAR(255) NOT NULL,
salt VARCHAR(32) NOT NULL
```

3. Validación de Datos de Teléfono

```
-- Sugerido
phone VARCHAR(15) NOT NULL CHECK (phone REGEXP '^[0-9+\\-\\s()]+$')
```

4. Índices para Optimización

```
-- Sugeridos
CREATE INDEX idx_person_lastname ON person(first_lastname, second_lastname);
CREATE INDEX idx_user_username ON user(username);
```

Mejoras de Diseño

1. Separación de Entidades

- Tabla separada para direcciones
- Tabla para tipos de teléfono
- Normalización adicional

2. Campos de Auditoría

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
created_by INT,
updated_by INT
```

3. Soft Delete

```
is_active BOOLEAN DEFAULT TRUE,
deleted_at TIMESTAMP NULL
```

Mejoras de Seguridad

1. **Encriptación de Contraseñas**
 2. **Validación de Entrada**
 3. **Escape de Caracteres Especiales**
 4. **Logs de Auditoría**
-

Conclusión

La base de datos `modelado_bd` representa un ejemplo sólido de diseño básico que implementa correctamente:

- **Principios de DDL** para definir estructura
- **Operaciones DML** para manipular datos
- **Integridad referencial** entre entidades
- **Buenas prácticas** fundamentales

El sistema cumple su propósito de gestionar usuarios y personas de manera relacionada, manteniendo la consistencia de datos y proporcionando una base sólida para futuras expansiones.

Documento generado para el curso de Modelado y Gestión de Bases de Datos - Semana 7, Sesión 1